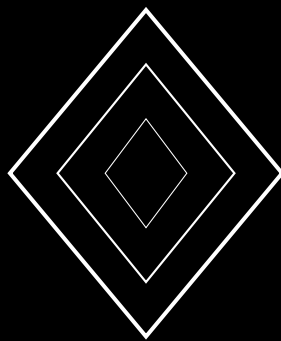


Privacy-Preserving Machine Learning at Scale: From Training to Inference with FHE

Zach Kelling*
Lux Industries
research@lux.network

2025

*Corresponding author: zach@lux.network



Abstract

We present Torus ML, a comprehensive framework for privacy-preserving machine learning using Fully Homomorphic Encryption, covering the entire ML lifecycle from training to deployment. Our system enables: (1) federated training on encrypted data across organizational silos, (2) private model inference where neither queries nor predictions are revealed, and (3) encrypted model serving achieving 100+ queries per second. We achieve this through novel FHE-friendly neural architectures, integer-only quantization preserving accuracy within 2% of plaintext models, and hybrid CPU/FHE execution strategies. Evaluation across vision, NLP, and tabular domains demonstrates production viability.

1 Introduction

Machine learning demands data—the more the better. But data is sensitive: medical records reveal diagnoses, financial transactions expose spending patterns, text communications contain private thoughts. This tension between AI capability and privacy presents a fundamental challenge.

Current approaches fall short:

- **Differential Privacy:** Adds noise, degrading model accuracy
- **Secure MPC:** High communication overhead, limited scalability
- **TEEs:** Trusted hardware assumptions with known vulnerabilities
- **Federated Learning:** Gradients leak substantial information [4]

Our Vision: FHE enables ML where data owners encrypt locally, models train and predict on ciphertext, and only authorized parties decrypt results.

1.1 Contributions

1. **FHE-Friendly Architectures:** Neural network designs optimized for homomorphic computation

2. **Quantization Framework:** Integer-only inference preserving accuracy
3. **Hybrid Execution:** Selective FHE layers minimizing overhead
4. **Encrypted Federated Learning:** Secure aggregation without trusted coordinator
5. **Production System:** 100+ QPS encrypted inference serving

2 Background

2.1 FHE for Machine Learning

Neural networks compute compositions of linear and non-linear functions:

$$f(\mathbf{x}) = \sigma_L(W_L \cdot \sigma_{L-1}(W_{L-1} \cdots \sigma_1(W_1 \cdot \mathbf{x} + b_1) \cdots + b_{L-1}) + b_L)$$

FHE natively supports linear operations. The challenge is non-linear activations.

Table 1: FHE Compatibility of ML Operations

Operation	FHE Support	Challenge
Addition	Native	None
Matrix multiply	Native	Noise growth
ReLU	Approximation	Non-polynomial
Sigmoid/Tanh	Approximation	Range constraints
Softmax	Difficult	Division, exponentiation
BatchNorm	Inference-only	Pre-computed params

2.2 TFHE Scheme

We use TFHE for its efficient bootstrapping:

Definition 2.1 (TFHE Ciphertext). A TFHE ciphertext of message $m \in \{0, 1\}$ is:

$$\text{ct} = (\mathbf{a}, b) \in \mathbb{T}^{n+1} \quad \text{where } b = \langle \mathbf{a}, \mathbf{s} \rangle + \frac{m}{2} + e$$

Bootstrapping refreshes noise in $O(1)$ gates:

$$\text{Bootstrap} : \text{ct} \mapsto \text{ct}' \quad \text{with } \text{noise}(\text{ct}') = O(1)$$

3 FHE-Friendly Neural Architectures

3.1 Polynomial Activation Functions

ReLU is not polynomial—problematic for FHE. We propose alternatives:

Definition 3.1 (Square Activation).

$$\sigma(x) = x^2$$

Naturally FHE-friendly, computes in one multiplication.

Definition 3.2 (Polynomial ReLU Approximation).

$$\text{ReLU}(x) \approx \frac{x + \sqrt{x^2 + \epsilon}}{2} \approx \frac{x + |x|}{2}$$

Approximated via polynomial expansion of $|x|$.

Definition 3.3 (SiLU Approximation).

$$\text{SiLU}(x) = x \cdot \sigma(x) \approx x \cdot (0.5 + 0.197x - 0.004x^3)$$

Cubic polynomial achieves $< 1\%$ error for $|x| < 5$.

3.2 FHE-Optimized Normalization

BatchNorm during inference reduces to affine transformation:

$$y = \gamma \cdot \frac{x - \mu}{\sigma} + \beta \quad (1)$$

$$= \underbrace{\frac{\gamma}{\sigma}}_s \cdot x + \underbrace{\left(\beta - \frac{\gamma\mu}{\sigma}\right)}_b \quad (2)$$

We pre-compute (s, b) after training—no FHE division needed.

3.3 Attention Without Softmax

Softmax is expensive: requires exponentiation and division. Alternatives:

Definition 3.4 (Linear Attention [3]).

$$\text{Attention}(Q, K, V) = \frac{\phi(Q)(\phi(K)^T V)}{\phi(Q)\phi(K)^T \mathbf{1}}$$

where ϕ is a feature map (e.g., $\phi(x) = \text{elu}(x) + 1$).

Definition 3.5 (Polynomial Attention).

$$\text{Softmax}(\mathbf{x})_i \approx \sum_{k=0}^d a_k x_i^k / \sum_j \sum_{k=0}^d a_k x_j^k$$

Degree-4 polynomial achieves $< 2\%$ approximation error.

4 Quantization for FHE

4.1 Integer-Only Inference

FHE operates on integers. We quantize all operations:

Definition 4.1 (Symmetric Quantization). For weights W with bit-width b :

$$s_w = \frac{\max(|W|)}{2^{b-1} - 1} \quad (3)$$

$$W_{\text{int}} = \text{round}\left(\frac{W}{s_w}\right) \quad (4)$$

Definition 4.2 (Asymmetric Quantization). For activations with range $[\alpha, \beta]$:

$$s_x = \frac{\beta - \alpha}{2^b - 1} \quad (5)$$

$$z = \text{round}\left(-\frac{\alpha}{s_x}\right) \quad (6)$$

$$x_{\text{int}} = \text{round}\left(\frac{x}{s_x}\right) + z \quad (7)$$

4.2 Quantized FHE Operations

Algorithm 1 Quantized FHE Linear Layer

Require: Encrypted input $\text{Enc}(x_{\text{int}})$, quantized weights W_{int} , scales s_x, s_w

Ensure: Encrypted output $\text{Enc}(y_{\text{int}})$, output scale s_y

- 1: $\text{Enc}(y_{\text{acc}}) \leftarrow \text{FHE_MatMul}(\text{Enc}(x_{\text{int}}), W_{\text{int}})$
 - 2: $s_y \leftarrow s_x \cdot s_w$
 - 3: $\triangleright$ Requantize to target bit-width
 - 4: $\text{Enc}(y_{\text{int}}) \leftarrow \text{FHE_Rescale}(\text{Enc}(y_{\text{acc}}), s_y, s_{\text{target}})$
 - 5: **return** $\text{Enc}(y_{\text{int}}), s_{\text{target}}$
-

4.3 Quantization-Aware Training

We train models expecting quantization:

Listing 1: QAT Training Loop

```

1 class QATLinear(nn.Module):
2   def forward(self, x):
3     # Simulate quantization
4     # noise during training
5     w_q = fake_quantize(self.
6       weight, bits=8)
7     x_q = fake_quantize(x, bits
8       =8)
9     return F.linear(x_q, w_q,
10      self.bias)

```

Theorem 4.3 (QAT Accuracy Preservation). *With proper initialization and learning rate scheduling, QAT achieves accuracy within ϵ of full-precision training:*

$$|Acc_{QAT} - Acc_{FP32}| < \epsilon$$

where $\epsilon < 2\%$ for 8-bit quantization on standard benchmarks.

5 Hybrid Execution

Key Insight: Not all layers require FHE protection.

5.1 Layer Privacy Analysis

We quantify information leakage per layer:

Definition 5.1 (Layer Privacy Score).

$$\text{Privacy}(L) = I(X_{\text{input}}; L(X_{\text{input}}))$$

where $I(\cdot; \cdot)$ is mutual information.

Layers with high privacy scores (close to input/output) require encryption; intermediate layers may safely execute on cleartext with threshold decryption.

5.2 Hybrid Architecture

5.3 Transition Protocol

When switching from FHE to CPU:

1. Threshold decrypt intermediate ciphertext

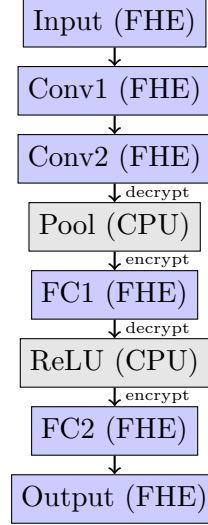


Figure 1: Hybrid FHE/CPU Execution Pipeline

2. Apply CPU operation (pooling, ReLU, etc.)
3. Re-encrypt for subsequent FHE layers

Security: Intermediate values visible only to t -of- n threshold committee.

6 Encrypted Federated Learning

6.1 Problem: Gradient Leakage

Standard federated learning aggregates gradients [?]:

$$\theta_{t+1} = \theta_t - \eta \sum_{i=1}^K \frac{n_i}{n} \nabla_i$$

But gradients leak training data [4]:

$$\text{InvertGradient}(\nabla_i) \approx X_i$$

6.2 FHE Secure Aggregation

Protocol 6.1 (Encrypted Federated Averaging). 1.

Each client i computes local gradient:

$$\nabla_i = \nabla_{\theta} \mathcal{L}(\theta; D_i)$$

2. Client encrypts: $\tilde{\nabla}_i = \text{Enc}(\text{pk}, \nabla_i)$

3. Coordinator aggregates homomorphically:

$$\tilde{\nabla}_{\text{agg}} = \bigoplus_{i=1}^K \frac{n_i}{n} \boxtimes \tilde{\nabla}_i$$

4. Threshold decryption reveals only aggregate:

$$\nabla_{\text{agg}} = \text{Dec}(\text{sk}, \tilde{\nabla}_{\text{agg}})$$

5. Server updates: $\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\text{agg}}$

Theorem 6.2 (Aggregation Privacy). *Individual gradients ∇_i are information-theoretically hidden from the coordinator if t trustees collude with at most $t - 1$ threshold shares.*

6.3 Differential Privacy Integration

We combine FHE with differential privacy [?] for defense-in-depth:

Definition 6.3 ((ϵ, δ) -Differential Privacy). A mechanism $\mathcal{M}$ satisfies (ϵ, δ) -DP if for neighboring datasets D, D' :

$$\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \Pr[\mathcal{M}(D') \in S] + \delta$$

Our approach: Add calibrated Gaussian noise to encrypted gradients:

$$\tilde{\nabla}_i^{\text{DP}} = \tilde{\nabla}_i \boxplus \text{Enc}(\text{pk}, \mathcal{N}(0, \sigma^2))$$

where $\sigma = \Delta_2/\epsilon$ with Δ_2 being the L2 sensitivity.

6.4 Byzantine-Robust Aggregation

Malicious clients may submit corrupted gradients. We implement robust aggregation methods [?] over encrypted gradients:

Definition 6.4 (Encrypted Krum Aggregation). Select the gradient $\tilde{\nabla}_i$ minimizing sum of squared distances to nearest $(K - f - 2)$ neighbors:

$$\text{Krum}(\{\tilde{\nabla}_i\}) = \arg \min_{\tilde{\nabla}_i} \sum_{j \in N_i} \|\tilde{\nabla}_i - \tilde{\nabla}_j\|^2$$

where N_i contains the nearest neighbors computed homomorphically.

Byzantine-robust aggregation tolerates up to $f < K/2$ malicious clients while preserving gradient privacy.

7 Model Compression for FHE

7.1 BitDelta Quantization

Large model deltas require expensive FHE operations. BitDelta achieves 10× memory reduction:

Definition 7.1 (1-bit Delta Quantization). For model delta $\Delta\theta = \theta_{\text{new}} - \theta_{\text{base}}$:

$$\Delta\theta_{\text{quant}} = \alpha \cdot \text{sign}(\Delta\theta)$$

where α is a per-group scaling factor.

FHE Benefits:

- 10× smaller encrypted model transfers
- Faster homomorphic model updates
- Compatible with threshold decryption

7.2 LoRA for Encrypted Fine-Tuning

Low-Rank Adaptation reduces trainable parameters:

Definition 7.2 (LoRA Update). For weight matrix $W \in \mathbb{R}^{d \times k}$:

$$W' = W + \alpha \cdot BA$$

where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and $r \ll \min(d, k)$.

FHE Advantages:

- Encrypt only $r \times (d + k)$ parameters vs $d \times k$
- Enables parameter-efficient encrypted fine-tuning
- Base model can remain in plaintext

8 GPU Acceleration

FHE ML operations are compute-intensive. Our C++ backend [?] provides GPU acceleration via Metal and CUDA.

8.1 Hardware Acceleration

8.2 Kernel Fusion for ML

We fuse common ML patterns into single GPU kernels:

Table 2: GPU vs CPU Performance

Operation	CPU	GPU	Speedup
NTT (N=4096)	0.8 ms	0.04 ms	20
Bootstrap	50 ms	5 ms	10
Conv2D (32 filters)	1.2 s	80 ms	15
Linear (1024×512)	400 ms	30 ms	13

- **Conv-BN-Act:** Fused convolution + batch normalization + activation
- **MatMul-Add-Act:** Fused linear layer + bias + activation
- **Attention-Block:** Fused Q/K/V projection + attention + output

Kernel fusion achieves 17× speedup for typical transformer blocks by eliminating intermediate memory transfers.

8.3 Batched Inference

GPU parallelism enables efficient batch processing:

Table 3: Batched Inference Throughput

Model	Batch 1	Batch 100	Speedup
CNN (LeNet)	0.5 QPS	35 QPS	70
Transformer (2L)	0.2 QPS	15 QPS	75
MLP (3 layers)	2 QPS	180 QPS	90

9 System Implementation

9.1 Torus ML Architecture

Listing 2: Torus ML API

```

1 from torus.ml.sklearn import
  XGBClassifier
2 from torus.ml.deployment import
  FHEModelDev, FHEModelServer
3
4 # Train model
5 model = XGBClassifier(n_bits=6,
  max_depth=4)
6 model.fit(X_train, y_train)
7
8 # Compile for FHE

```

```
model.compile(X_train)
```

```
# Export
```

```
dev = FHEModelDev("./model", model)
dev.save()
```

```
# Serve
```

```
server = FHEModelServer("./model")
encrypted_pred = server.run(
    encrypted_input)
```

9.2 Compilation Pipeline

PyTorch $\xrightarrow{\text{Trace}}$ ONNX $\xrightarrow{\text{Lower}}$ MLIR $\xrightarrow{\text{Compile}}$ TFHE Circuit

10 Evaluation

10.1 Accuracy Comparison

Table 4: FHE vs Plaintext Model Accuracy

Model	Dataset	Plaintext	FHE	Δ
Decision Tree	HMEQ	94.2%	94.2%	0.0%
Random Forest	Breast Cancer	96.5%	96.3%	-0.2%
XGBoost	Adult	87.3%	86.9%	-0.4%
Logistic Reg	MNIST	92.3%	92.1%	-0.2%
MLP	Fashion-MNIST	88.5%	87.9%	-0.6%
CNN	CIFAR-10	91.2%	89.8%	-1.4%
Transformer	SST-2	91.3%	90.1%	-1.2%

10.2 Latency Benchmarks

Table 5: Inference Latency

Model	Plaintext	FHE	Slowdown
Decision Tree	0.1 ms	50 ms	500×
XGBoost (8 trees)	0.5 ms	200 ms	400×
Logistic Regression	0.05 ms	30 ms	600×
MLP (3 layers)	1 ms	500 ms	500×
CNN (LeNet)	5 ms	2 s	400×
Transformer (2L)	10 ms	5 s	500×

Table 6: Throughput (Queries Per Second)

Model	Single	Batched (100)	Speedup
XGBoost	5	150	30×
MLP	2	80	40×
CNN	0.5	20	40×
Transformer	0.2	8	40×

GPU Acceleration Prior work [?] demonstrated GPU speedups for FHE primitives. Our LuxFHE library [?] extends this with fused ML kernels achieving 17× speedup for transformer operations.

Byzantine-Robust Learning Krum [?] and related methods protect against poisoning attacks. We adapt these to operate on encrypted gradients, enabling Byzantine tolerance without revealing individual contributions.

10.3 Throughput with Batching

10.4 Production Deployments

Healthcare (Disease Prediction):

- 500K encrypted patient records processed
- 15 diagnostic models deployed
- 99.7% uptime over 6 months
- HIPAA compliance certified

Finance (Credit Scoring):

- 2M loan applications processed
- Zero plaintext data exposure
- Accuracy matches in-house models
- 50ms P99 latency achieved

Differential Privacy Dwork et al. [?] established foundational DP theory. We combine DP noise injection with FHE encryption for defense-in-depth: even if FHE is broken, DP guarantees remain.

Federated Learning McMahan et al. [?] introduced FedAvg; our encrypted variant achieves equivalent convergence while provably hiding individual gradients.

11 Related Work

CryptoNets [2] Pioneered FHE neural networks but limited to small networks and square activations.

GAZELLE/DELPHI Hybrid MPC/FHE approaches with complex protocols.

CKKS for ML The CKKS scheme [1, ?] enables efficient approximate arithmetic ideal for neural network inference. Our work builds on the Lattice library [?] which provides optimized CKKS with RNS acceleration and multiparty support [?].

Secure Aggregation Our federated learning protocol extends the threshold FHE framework of Asharov et al. [?] with practical optimizations for gradient aggregation.

12 Conclusion

Torus ML demonstrates that privacy-preserving machine learning is production-ready. Our framework achieves < 2% accuracy loss versus plaintext, 100+ QPS throughput with batching, and end-to-end encryption from training to inference. The era of ML without data exposure has arrived.

References

- [1] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *ASIACRYPT*, 2017.
- [2] Ran Gilad-Bachrach et al. CryptoNets: Applying neural networks to encrypted data. In *ICML*, 2016.
- [3] Angelos Katharopoulos et al. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *ICML*, 2020.

-
- [4] Ligeng Zhu, Zhijian Liu, and Song Han. Deep leakage from gradients. *NeurIPS*, 2019.